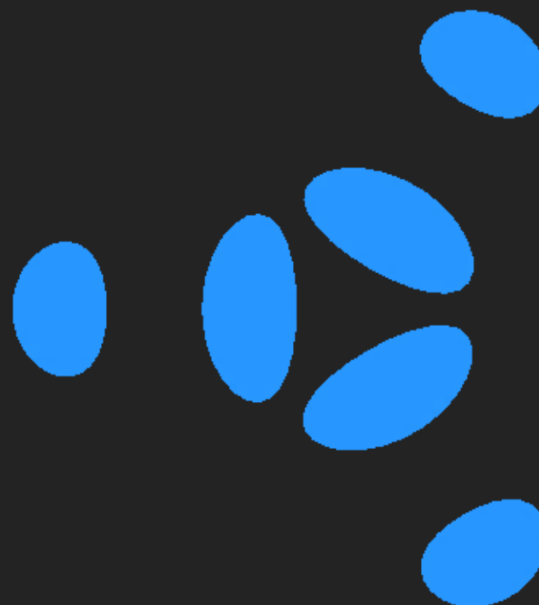




***COMEÇAREMOS
EM BREVE!***





especialização

Análise de Dados em Redes de Telecom

Prof. Dr. Jonas Augusto Kunzler

Open RAN:

Redes Abertas e Tecnologias Emergentes

 **Bloco 1 — 18:30–20:00**

Data lakes, warehouses e aprofundamento Open RAN

Disciplina 9: Análise de Dados em Redes de Telecom

Aula 02: Arquiteturas de dados em telecom

Data: 06/08/2026 | Duração: 3h30 | Carga horária total: 24 h

Onde persistir PM/FM/KPM — **lake vs warehouse**, SQLite/MySQL/MongoDB/Firebase

+ **InfluxDB** (O-RAN SC Non-RT / RAN PM) e consultas no terminal



Calendário do curso — 6 encontros (24h)

Encontro	Data	Duração	Conteúdos
Aula 01	04/08/2026	3h30	Fontes + projeto integrador
Aula 02	06/08/2026	3h30	Lakes/warehouses + DBs + caso preliminar
Aula 03	08/08/2026	5h	EDA/ETL + Checkpoint 1
Aula 04	25/08/2026	3h30	KPIs/KQIs + Checkpoint 2
Aula 05	27/08/2026	3h30	Capacidade + checkpoint técnico
Aula 06	29/08/2026	5h	Apresentações e defesa

Acordos Didáticos

- Projeto integrador (50%) · Plataforma individual (30%) · Engajamento e checkpoints (20%)
- Presença: início, intervalos e encerramento
- Clone do repositório do curso: <https://github.com/jakunzler/cesar-school-repo>

Cronograma Previsto — Aula 02

- 18:30–19:20 — Retomada (paradigmas · tipos) + Lake · warehouse · lakehouse · lote/ streaming
- 19:20–20:00 — Aprofundamento Open RAN: escalas · OLTP/TSDB/OLAP · O1×E2 · dimensional
- 20:00–20:10 — Intervalo
- 20:10–21:00 — DBs + terminal: SQLite · MySQL · Redis · Influx · Mongo/Firebase
- 21:00–21:10 — Intervalo
- 21:10–22:00 — Atividade arquitetura + checkpoint G1–G7

Objetivos — Aula 02

Objetivo central

Escolher arquitetura de armazenamento e o tipo de banco adequado ao caso telco/lab

Ao final da aula

Consultar SQLite/MySQL/MongoDB; mapear Influx/O1/E2; propor arquitetura DW/Lake do projeto

Texto base e referências de arquitetura

- **Tripathi** §2.2–2.6 — SMO, O-Cloud/O2, exposição de dados (R1)
- **Wong** Caps. 3–4 — SMO/O-Cloud e cloudificação (localidade edge/regional)
- **Databricks** — lake vs warehouse e convergência lakehouse (schema-on-read vs schema-on-write)
- **Inmon** / **Kimball** (conceitos) — DW corporativo vs modelagem dimensional
- **O-RAN SC Non-RT RAN PM** — VES → **Kafka** → Influx Logger → **InfluxDB** (rApps)
- Lab: JSONL + SQLite · Core OAI: MySQL · Near-RT OSC: Redis `ric_dbaas` (RNIB)

Conexão com a Aula 01

- Fontes mapeadas (RAN/core/transporte/terminal) precisam de **onde persistir**.
- Pipeline: **fonte** → **ingestão** → **armazenamento** → **qualidade** → **consulta**.
- Hoje: arquitetura + **bancos concretos** + **comandos de terminal**.
- Checkpoint: caso G1–G7 + hipótese + **lugar de persistência**.

Glossário rápido

Termo	Ideia em uma linha
Schema-on-read	Interpreta o schema na consulta (típico do lake)
Schema-on-write	Valida/transforma ao carregar (típico do warehouse)
ELT	Extract → Load → Transform no destino
OLTP	Transações operacionais (muitas escritas curtas)
OLAP	Consultas analíticas (agregações, histórico)
TSDB	Time-series DB (ex.: InfluxDB) — métricas indexadas no tempo
RNIB / SDL	Inventário Near-RT (Redis <code>ric_dbaas</code>) — não é histórico PM
BaaS	Backend-as-a-Service (ex.: Firebase)

Retomada — linguagens, paradigmas e tipos

- Antes de lake/warehouse: alinhar **como pensamos o programa** e o que é um **dado tipado**.
- Ponte com áreas do conhecimento: **paradigmas de programação e modelagem de dados** (atributos → colunas SQL).
- Na análise telco: o mesmo conceito (ex.: vazão) aparece como `float` em Python, coluna `REAL` no SQLite e campo JSON no bronze.

Paradigmas de programação (visão rápida)

Paradigma	Ideia central	Âncora no lab
Imperativo	Sequência de comandos que alteram estado	Script que coleta KPM linha a linha
Estruturado	Sequência, seleção, iteração + procedimentos	Funções <code>export_*</code> / <code>run_*</code>
Funcional	Funções puras, composição, imutabilidade	Pipelines de transformação (map/filter)
Orientado a objetos	Dados + comportamento encapsulados	Classes de amostra, cliente DB, decisão

Paradigma = forma de **organizar a solução**, não só a sintaxe da linguagem. 13

Procedimentos × dados (e o salto da OO)

- **Estruturado:** procedimentos (o que fazer) e estruturas de dados (o que guardar) ficam *separados* — reuso e manutenção ficam mais difíceis em sistemas grandes.
- **OO:** o objeto une **estado** (atributos) e **comportamento** (métodos); comunicação por mensagens.
- **Em análise de dados:** ainda há separação útil — *dados persistidos* (JSONL/SQLite) vs *código que transforma* (notebook/script). OO organiza o código; o lake/warehouse organiza o armazenamento.

Procedimentos × dados (e o salto da OO)

Exemplo:

```
// Estruturado: dado à parte + função
float prb = 0.82;
const char* classificar(float p) {
    return (p > 0.7) ? "alto" : "ok";
}
```

```
# OO: dado + regra no mesmo conceito
class AmostraKpm:
    def __init__(self, prb): self.prb = prb
    def classificar(self): return "alto" if self.prb > 0.7 else "ok"
```

Tipos básicos — programação e modelagem

Conceito	Python	SQL (ex.)	Exemplo telco
Inteiro	<code>int</code>	<code>INTEGER</code> / <code>INT</code>	contagem de UEs, índice de amostra
Real	<code>float</code>	<code>REAL</code> / <code>FLOAT</code>	vazão (Mbps), PRB used ratio
Texto	<code>str</code>	<code>TEXT</code> / <code>VARCHAR</code>	cell_id, fase (<code>baseline</code>)
Lógico	<code>bool</code>	<code>BOOLEAN</code> / 0·1	alarme ativo, flag de anomalia

Tipos básicos — programação e modelagem

Conceito	Python	SQL (ex.)	Exemplo telco
Nulo / ausência	<code>None</code>	<code>NULL</code>	métrica não reportada no intervalo
Coleção	<code>list</code> / <code>dict</code>	tabela / JSON	série temporal; payload KPM

Modelagem (ER): atributo *simples/atômico* → coluna tipada; composto → várias colunas; multivalorado → tabela auxiliar ou JSON.

Tipos de dados e formatos (armazenamento)

Tipo	Exemplo telco	Formatos comuns
Estruturado	KPI agregado, inventário	CSV, Parquet, SQL
Semiestruturado	Alarme FM, amostra KPM	JSON, JSONL, Avro
Não estruturado	Log textual, captura	Texto, PCAP (fora do lab)

Ponte: tipos básicos (`int` / `float` / `str`) vivem *dentro* desses formatos — schema-on-write tipa na carga; schema-on-read tipa na consulta.

Lab / projeto: JSONL (bronze) + SQLite (curado). Parquet/Avro: típicos em lake/warehouse de operador.

Da tipagem ao depósito — contextualização

- Até agora: **paradigmas** (como organizar o código), **tipos básicos** (`int` / `float` / `str`) e **formatos** (estruturado · semiestruturado · não estruturado).
- Surge a pergunta: onde esses valores **vivem no tempo**, com volume e múltiplos consumidores?
- Resposta em duas famílias clássicas: **Data Warehouse** (curado, analítico) e **Data Lake** (bruto/flexível) — depois, *lakehouse* como convergência.
- O código tipa e transforma; o depósito define **quando** o schema é imposto e **para quem** o dado serve.

Por que não basta o arquivo do script?

- **Retomando:** procedimentos × dados — o script processa; o depósito *sobrevive* à execução.
- Várias fontes (RAN, core, logs) → vários formatos → precisa de **integração** e histórico.
- Vários leitores (analista, painel, ML, auditoria) → precisa de **contrato semântico** ou de exploração controlada.
- No lab: `kpm.jsonl` (flexível) e SQLite tipado (consultável) já antecipam lake × warehouse em escala reduzida.

Data warehouse — o que é

- Repositório orientado a **análise e decisão** (não ao caminho crítico de controle da RAN).
- Dados **integrados**, tipicamente com **schema-on-write**: tipagem e regras na carga (DDL, ETL).
- Otimizado para leitura analítica (**OLAP**): agregações, histórico, drill-down — não para milhares de transações curtas (**OLTP**).
- Em telecom: KPIs oficiais, SLA, capacidade, relatórios regulatórios / Network Operations Center (NOC) de gestão.
- Analogia no lab: tabelas SQLite tipadas + agregações por fase (`baseline` / `stress`).

Quatro traços clássicos do DW — Inmon

TRAÇO	SIGNIFICADO	EXEMPLO TELCO
Temático	Organizado por assunto de negócio	domínio “capacidade RAN”, não dump bruto de E2
Integrado	Unifica fontes com nomes/unidades coerentes	PRB e Thp alinhados entre células/fornecedores
Variável no tempo	Mantém histórico (séries, snapshots)	KPI por célula × 15 min ao longo de meses
Não volátil	Carrega e consulta; não “edita” o passado operacional	append/correção via nova carga versionada

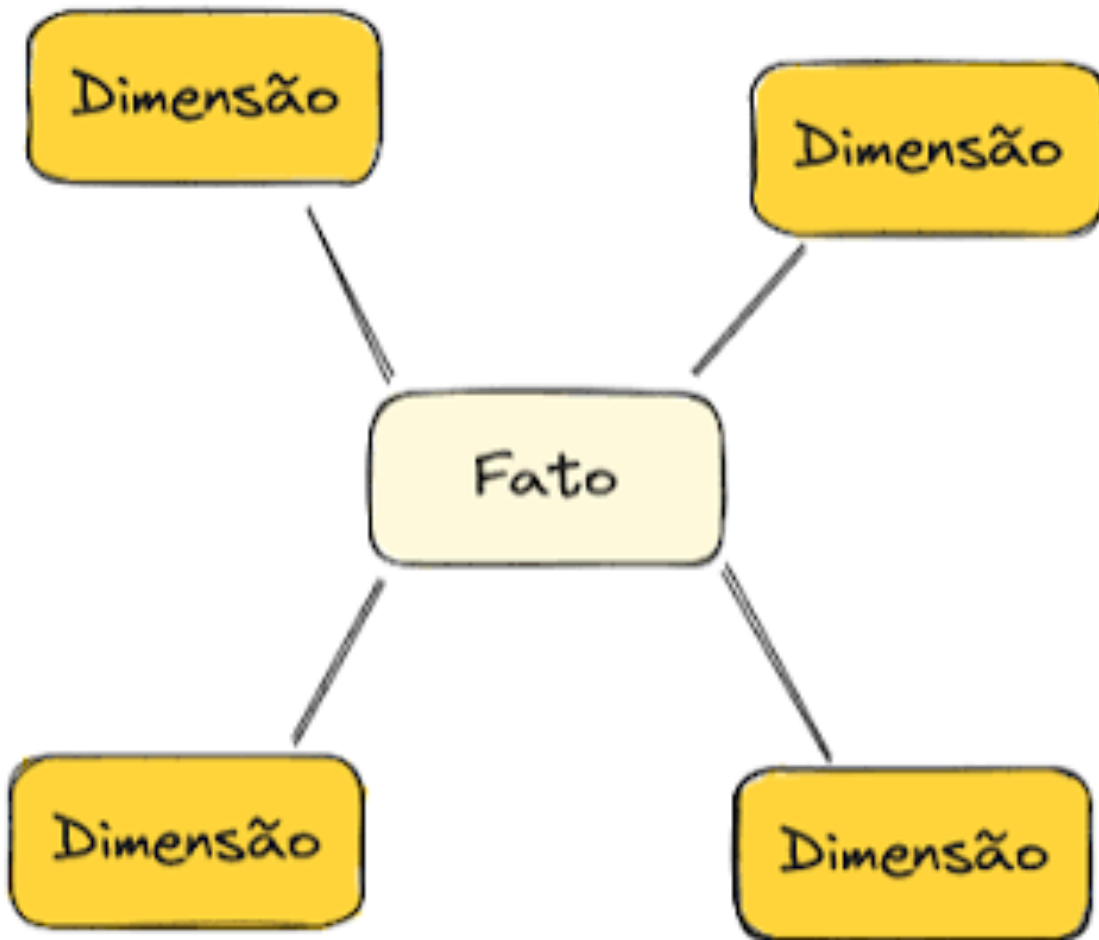
Definição clássica de **Bill Inmon** para o Data Warehouse corporativo (Enterprise Data Warehouse - EDW).

Modelagem dimensional — abordagem Kimball

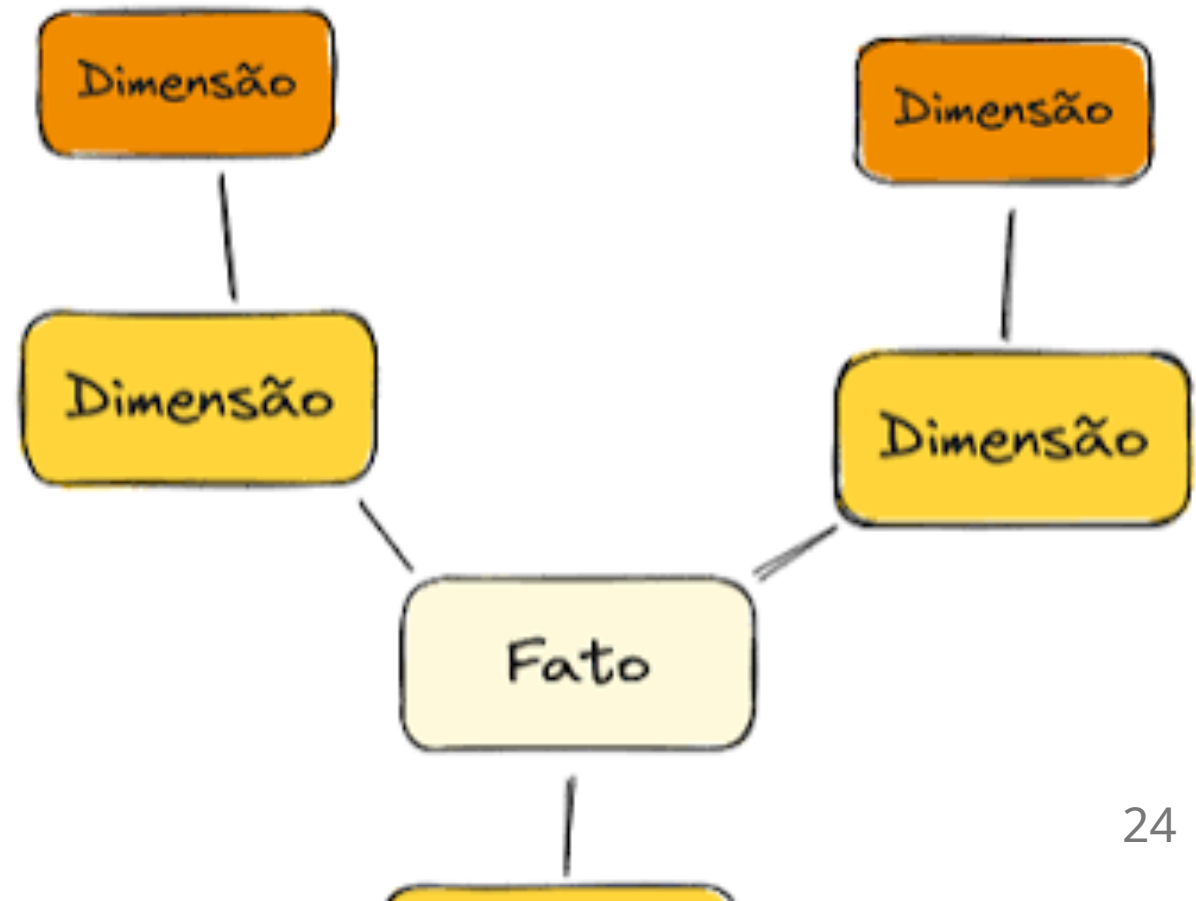
- **Ralph Kimball:** DW pensado a partir de *como o negócio pergunta* — medidas no contexto.
- **Tabela fato:** eventos/medidas numéricas (ex.: Thp, PRB, UEs) numa granularidade declarada.
- **Tabelas dimensão:** contexto da análise (tempo, célula, site, slice, fornecedor...).
- **Padrões de schema:** *estrela* e *floco de neve* (snowflake) — formas de ligar fato e dimensões.
- **Caminho típico:** data marts / estrelas → integração via *dimensões conformadas* ("bus").

Esquema estrela x floco de neve

Star Schema



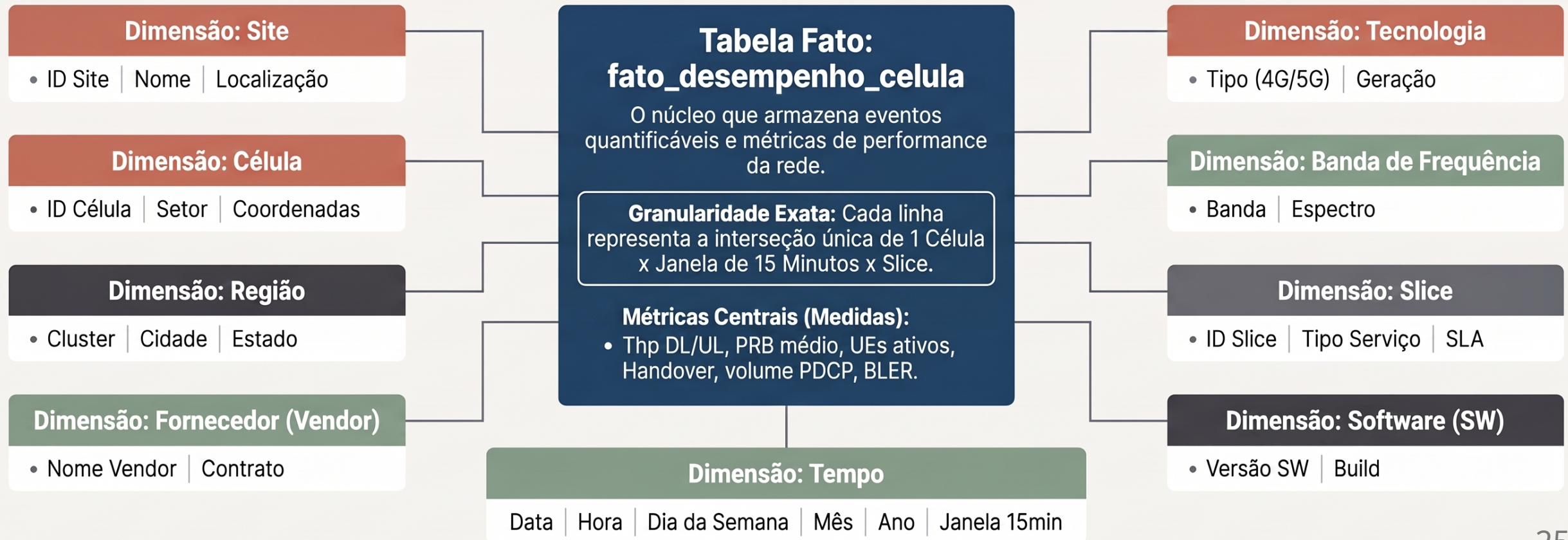
Snowflake



Esquema estrela no contexto O-RAN

Modelagem Dimensional: Schema Estrela para KPIs de RAN

Arquitetura de Dados (Star Schema) Otimizada para Análise de Desempenho de Redes de Acesso Rádio (RAN)



Estrela × floco — o que muda

	ESTRELA (STAR)	FLOCO (SNOWFLAKE)
Dimensões	Uma tabela por dimensão (desnormalizada)	Dimensões normalizadas em subníveis
Joins	Poucos — fato ↔ dimensão	Mais joins (dimensão ↔ subdimensão)
Consulta	Simples para BI / analista	Mais rígida; menos redundância
Ex. telco	<code>dim_celula</code> com site e região embutidos	<code>dim_celula</code> → <code>dim_site</code> → <code>dim_regiao</code>

Ambos são **modelos dimensionais** (família Kimball). Estrela é o default didático e o mais comum em marts.

Como estrela/floco se ligam a Inmon e Kimball

- Não são “escolas” rivais a Inmon/Kimball — são *padrões de schema* do lado dimensional.
- **Kimball:** estrela (e, se preciso, floco) é o coração do desenho — marts + dimensões conformadas. Preferência histórica pela **estrela** (simplicidade de consulta).
- **Inmon:** o EDW costuma ser **normalizado (3NF)**; estrela/floco aparecem nos **data marts derivados** do EDW (camada de consumo), não como modelo do núcleo corporativo.
- **Resumo:** Inmon/Kimball = *estratégia de construção*; estrela/floco = *forma do modelo analítico* que o usuário consulta.

Inmon × Kimball — duas abordagens

	INMON	KIMBALL
Ênfase	DW corporativo integrado (EDW)	Modelagem <i>dimensional</i> para análise
Caminho típico	Top-down: EDW → data marts derivados	Bottom-up: marts / estrela → “bus” com dimensões conformadas
Núcleo	Muitas vezes 3NF (não é estrela)	Fato + dimensões (estrela; floco se necessário)
Estrela/floco	No <i>mart</i> (consumo), após o EDW	Desde o desenho do mart / bus
Pergunta guia	Como integrar a empresa numa fonte analítica?	Como o negócio consulta medidas por contexto?

Mesmo fim (análise); **estratégias diferentes** — estrela/floco são ferramentas do plano dimensional.

Data lake — o que é

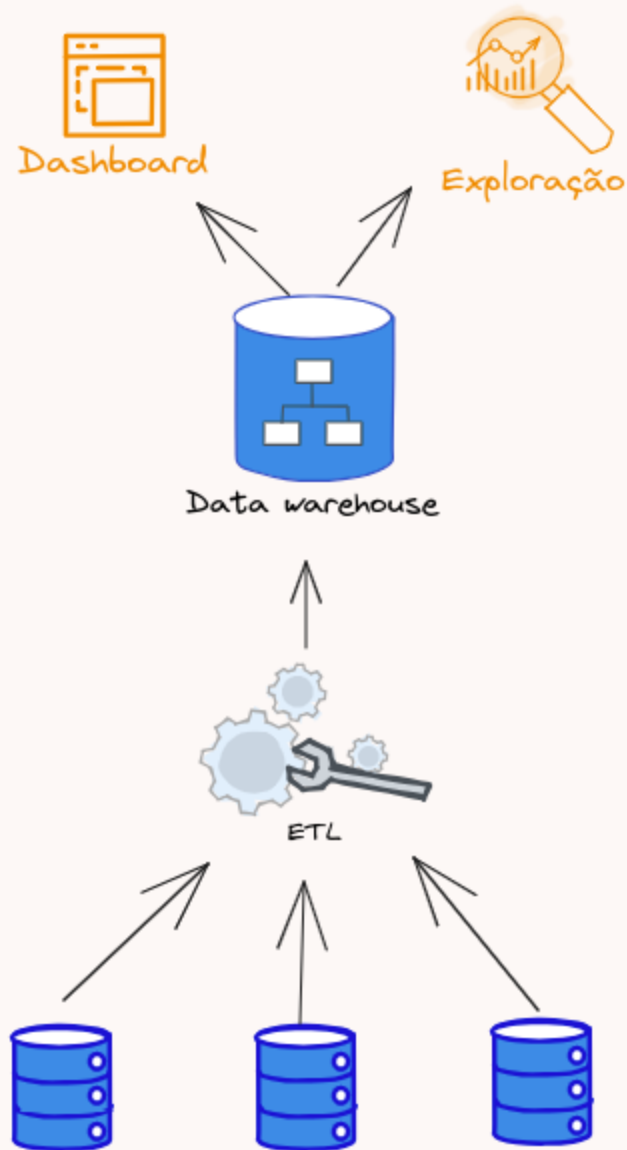
- Repositório de dados **brutos ou pouco transformados**, em escala — arquivos, objetos, logs, telemetria.
- Tipicamente **schema-on-read**: o formato chega primeiro; o schema se aplica na consulta/exploração.
- Acolhe o que vimos em formatos: JSONL, texto, dumps PM, eventualmente PCAP (fora do lab).
- Forte para exploração, reproprocessamento e treino ML; frágil sem catálogo → **data swamp**.
- Em telecom: histórico PM bulk, traces, configs (com ACL). Lab: `kpm.jsonl` / logs de Indication.

Lake e warehouse — a mesma pergunta, respostas diferentes

PERGUNTA	DATA WAREHOUSE	DATA LAKE
Quando tipa?	Na escrita (schema-on-write)	Na leitura (schema-on-read)
O que guarda?	Curado, semântica estável	Bruto / flexível / multi-formato
Quem consome?	BI, KPI oficial, auditoria	Ciência de dados, investigação, reprocessamento
Risco principal	Rigidez / atraso na mudança de schema	Pântano sem metadados e linhagem
Âncora lab	SQLite + agregações	JSONL bronze

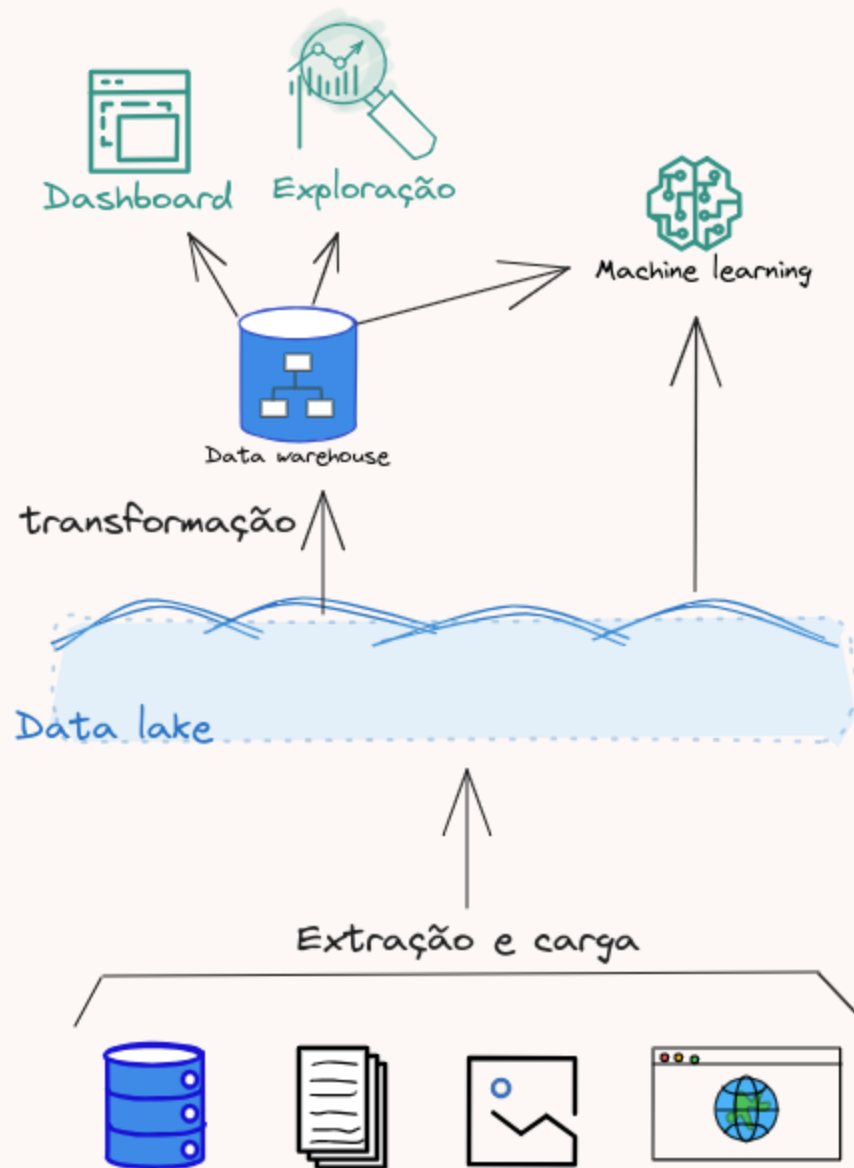
Não são rivais absolutos — em operadores costumam coexistir; o lakehouse tenta unificar planos.

DATA WAREHOUSE



Bases de dados relacionais (estruturados)

DATA LAKE



Dados estruturados e não estruturados

Lakehouse — convergência (visão rápida)

- Mantém o **bruto** (estilo lake) e expõe **tabelas confiáveis** (estilo warehouse) no mesmo ecossistema.
- Ideia: SQL, versionamento e qualidade sobre armazenamento de objetos (ex.: Parquet + camada de tabela).
- Útil quando se quer reprocessar fórmulas de KPI e servir BI/ML sem dois silos totais.
- No curso: conceito de referência; no lab offline o “mini-lakehouse” é JSONL + SQLite + notebook.

Tríptico Comparativo – Data Lake × Data Warehouse × Data Lakehouse

	 DATA LAKE	 DATA WAREHOUSE	 DATA LAKEHOUSE
Tipos de Dados	Dados Brutos , Diversos (Arquivos, JSON, Binário)	Dados Estruturados (Kimball, Schema Estrela)	Híbrido : Estruturado, Semiestruturado, Bruto
Schema	On-read (Flexível)	On-write (Rígido)	Híbrido , Evolução de Schema
Governança	Média , Risco de “Pântano de Dados”	Alta , Controles Estritos	Alta (Integrada, suporte ACID/Transações)
Reprocessamento	Sim , Total e Parcial	Não (Muito Complexo) Cargas de ETL Fixas	Sim , Suporta Evolução de Dados
Usuários Típicos	Cientistas de Dados, Analistas de Investigação	Analistas de BI, Gestores	Todos (Cientistas, Analistas, Engenheiros)
Casos Open RAN (Assurance vs Investigação)	Investigação de Falhas e Logs Complexos	BI Histórico, Painéis de SLA	BI de KPI e Treinamento de ML (rApps)

ACID=Atomicity Consistency Isolation Durability, **BI**=Business Intelligence, **ML**=Machine Learning, **SLA**=Service Level Agreement, **ETL**=Extract Transform Load, **KPI**=Key Performance Indicator, **rApp**=r-Application (componente do RIC Non-RT), **JSON**=JavaScript Object Notation

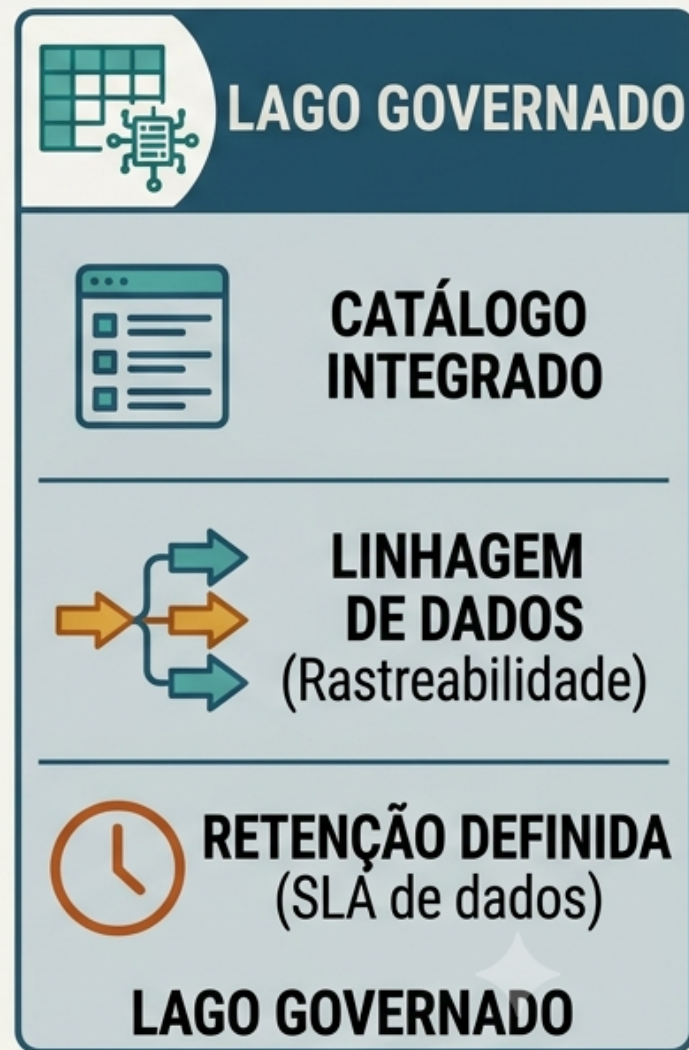
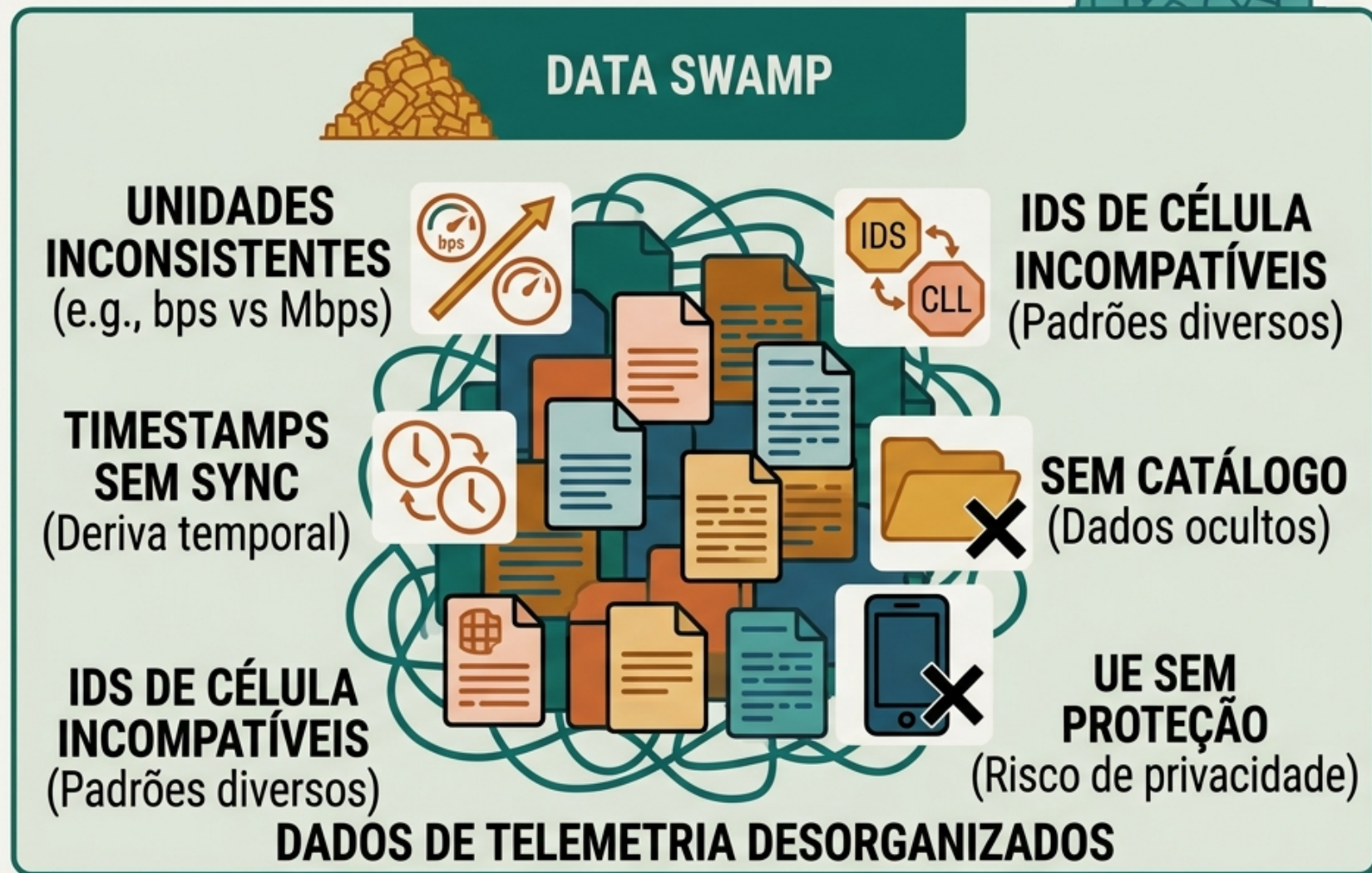
DATA SWAMP

TELECOM

VOLUME



OBSERVABILIDADE



A partir da organização dos dados

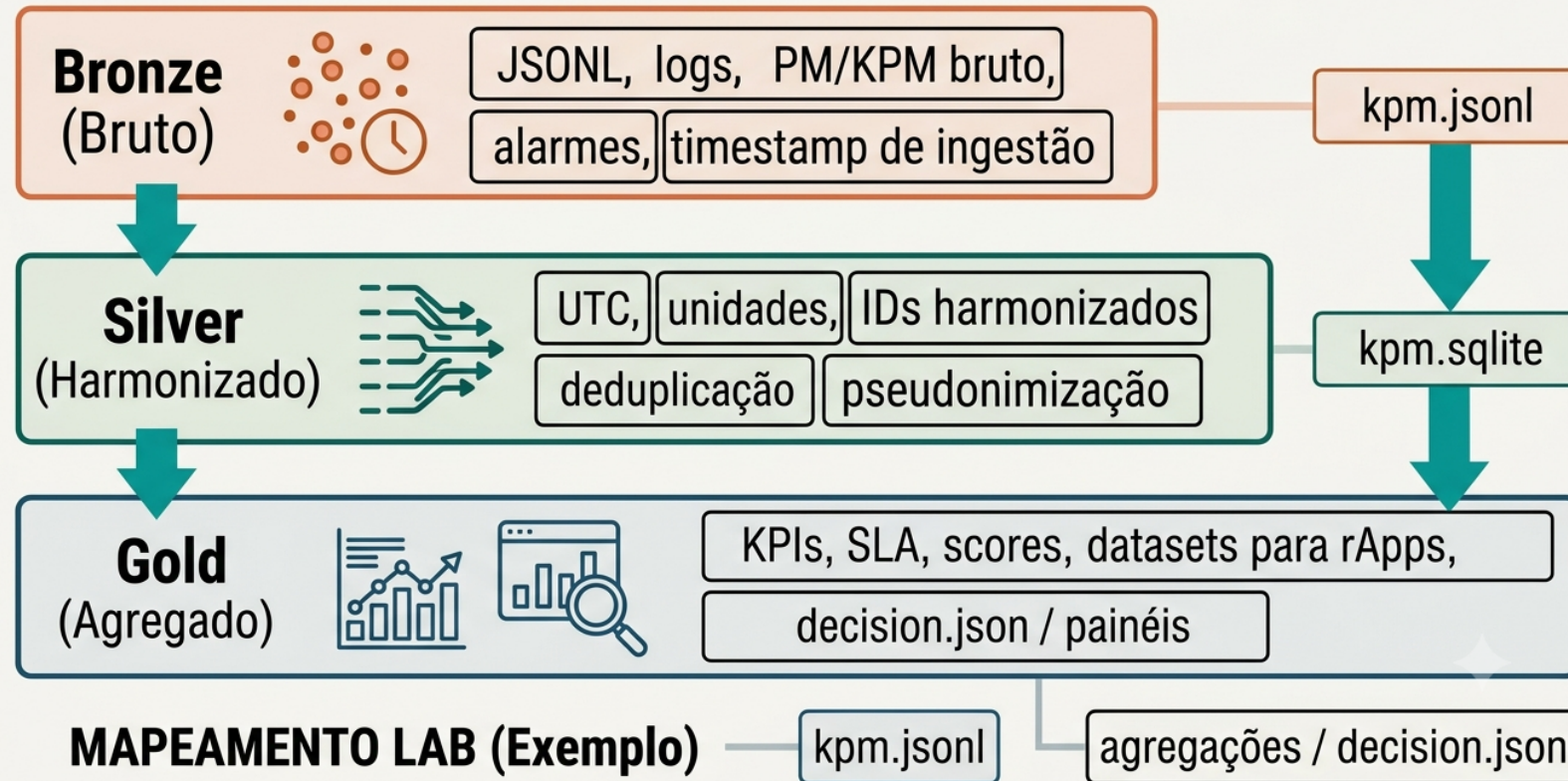
- **Lote × streaming** — como o dado chega ao depósito.
- **Bronze · silver · gold** — como organizar qualidade sem confundir com norma O-RAN.
- **Bloco Open RAN** — onde DW/Lake se encaixam nas escalas Near-RT / Non-RT (sem substituir E2/A1).
- **Bancos concretos** — SQLite, MySQL, Influx, Redis como instâncias das famílias.

Lote vs. streaming

Lote	Streaming
Agregações diárias/horárias de PM	Alarmes e anomalias em tempo quase real
Backfill e reproprocessamento	Janelas (tumbling, sliding)
PM O1 bulk (arquivo)	O1 streaming · E2 KPM near-RT
Bom para treino offline	Bom para NOC / closed-loop

Camadas bronze · silver · gold

“Medallion – convenção, não norma O-RAN”



Camadas bronze · silver · gold

- **Bronze:** bruto — JSONL KPM, dumps PM, alarmes como chegaram.
- **Silver:** limpo e tipado — UTC, unidades, chaves de célula/UE.
- **Gold:** indicadores curados — KPIs/KQIs para painel e decisão.
- *Não é norma O-RAN; é analogia de organização (medallion).*

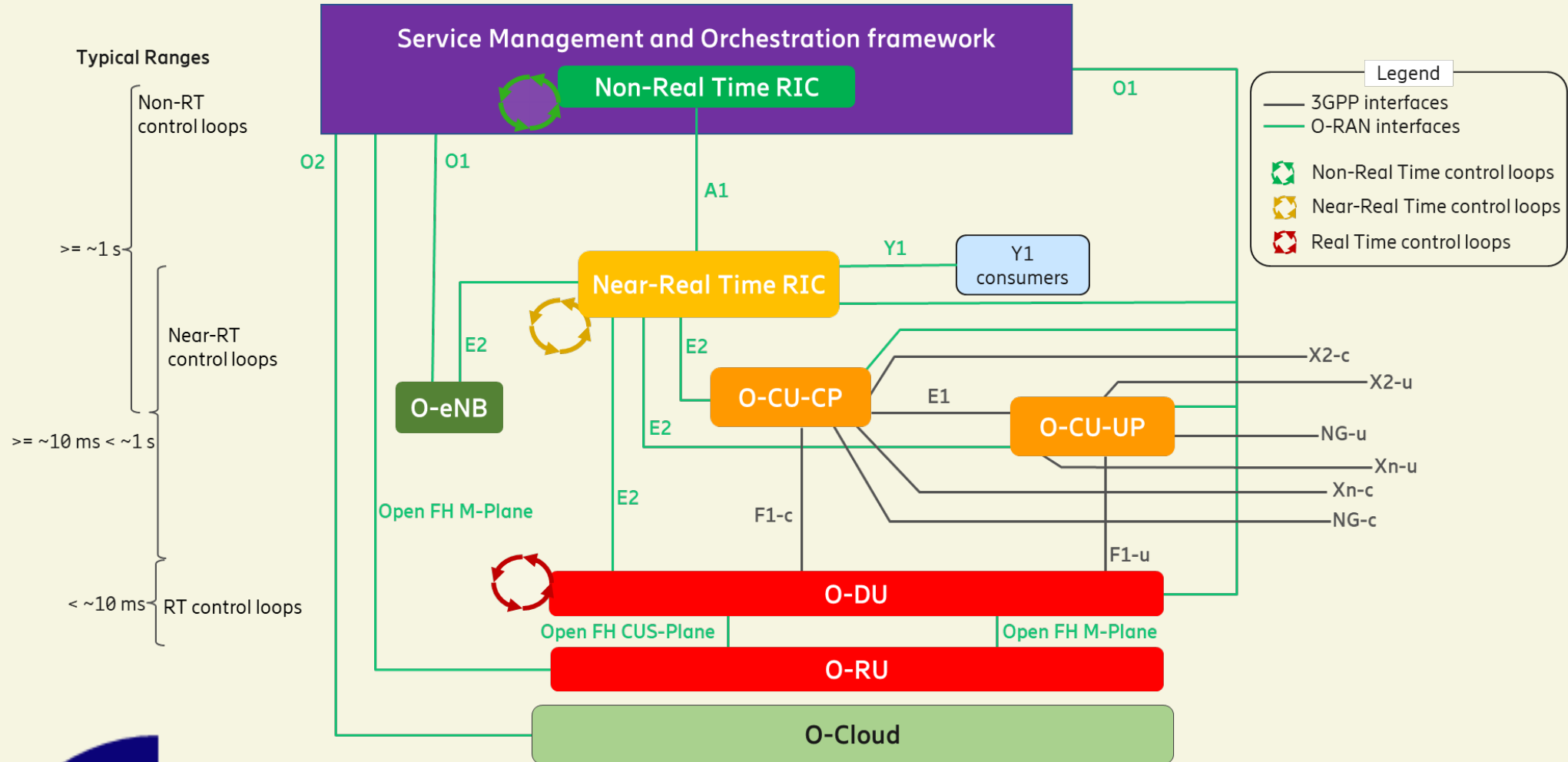
Lab: JSONL → SQLite → `decision.json` .



Bloco 1 — arquitetura analítica Open RAN

Aprofundamento (DW · Lake · interfaces · governança)

Arquitetura O-RAN e loops de controle



Escalas de decisão (orientação central)

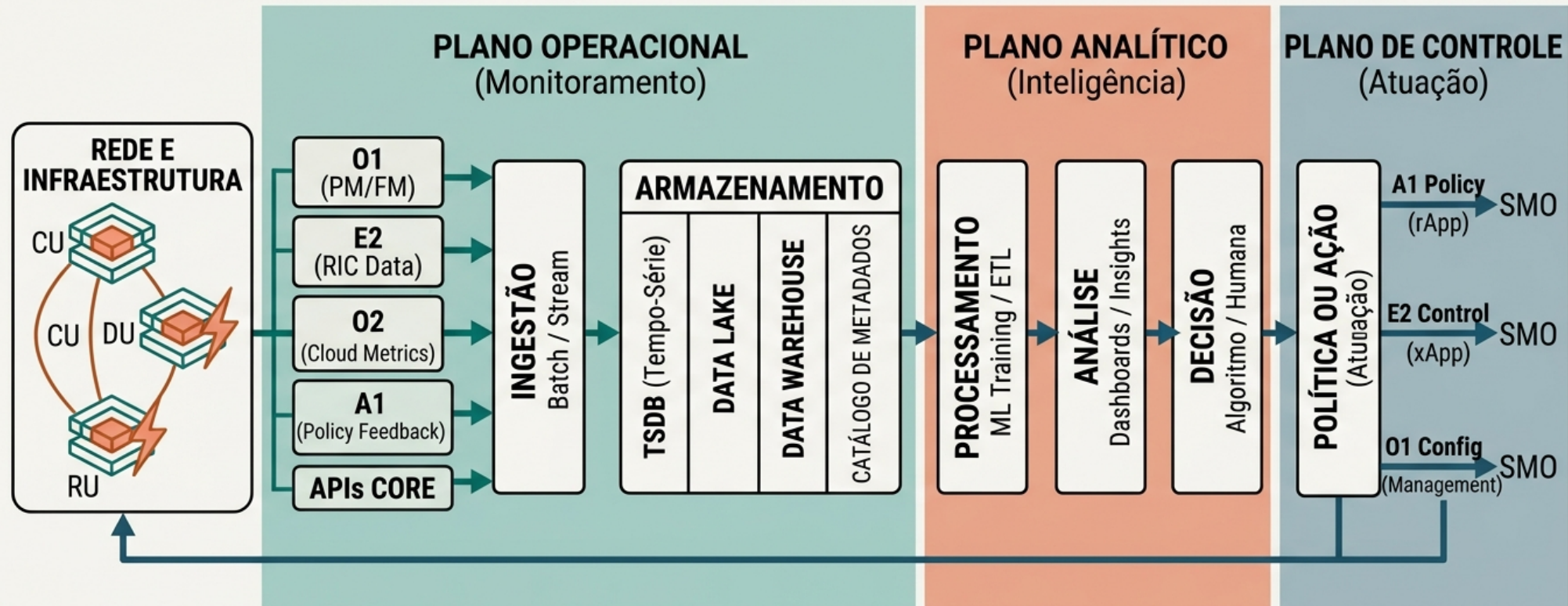
Escala	Onde	Papel típico do dado
Tempo real / muito baixa latência	PHY/MAC, loops locais	Controle interno da RAN
Near-RT	near-RT RIC / xApps (E2)	Observação e controle em segundos ou menos
Non-RT	non-RT RIC / rApps, SMO	Histórico, políticas A1, treino
Offline	lab, auditoria, pesquisa	Reprodutibilidade, benchmarking

Problema motivador — uma rede Open RAN

- **RAN:** O-RU · O-DU · O-CU · múltiplas células e slices
- **Controle:** near-RT RIC · non-RT RIC · SMO · O-Cloud
- **Core 5G:** AMF · SMF · UPF · assinantes/sessões
- **Dados:** PM/KPM · alarmes · logs · traces · config · QoS/QoE · inventário

Como integrar, armazenar, governar e analisar sem comprometer loops de controle?

Ciclo de dados em Open RAN



01=Interface de Gestão, **E2**=Interface de Controle Near-RT
A1=Interface de Política Non-RT, **02**=Interface O-Cloud
SMO=Service Management & **TSDB**=Time-Series Database
Legenda: "Nem toda medição gera ação imediata."

Ciclo de dados em Open RAN

Rede/infra → coleta → ingestão → armazenamento → processamento
→ análise → decisão → política/ação

Etapa	Exemplos
Coleta	O1 · E2 · A1 · O2 · APIs Core · exporters
Ingestão	brokers · collectors · batch/stream
Store	TSDB · Lake · Warehouse · catálogo
Consumo	NOC · rApps · xApps · ML · assurance

Nem toda medição gera ação imediata; nem toda ação depende de histórico.

Tipos de dados em Open RAN

Classe	Exemplos
Desempenho (PM/KPM)	PRB, throughput, latência, BLER, handovers, UEs
Falha (FM)	alarmes, sync, falhas HW/SW/interface
Configuração (CM)	célula, potência, vizinhanças, slices, VNF
Contexto	topologia, inventário, versão SW, capacidade
Sessão / UE	QoS Flow, PDU Session, mobilidade (pseudonimizado)

PM = Performance Management · FM = Fault Management ·

CM = Configuration Management · KPM = Key Performance Measurement

Operacional × analítico

	Operacional	Analítico / treino
Finalidade	servir a NF / controle	histórico, KPI, ML
Latência	baixa	segundos...horas (ok)
Retenção	curta / estado	semanas-anos
Consulta	pontual	agregações pesadas
Exemplos	MySQL Core · Redis RNIB · config SMO	Warehouse · Lake · Parquet
TSDB	Influx/Prometheus — telemetria recente (meio-termo operacional-analítico)	

OLTP x TSDB x OLAP: COMPARAÇÃO DE BANCOS DE DADOS EM OPEN RAN



OLTP

(Processamento de Transações Online)

Finalidade: Configuração de rede, estado de sessão.

Padrão de Escrita: Múltiplas escritas pequenas e rápidas.

Retenção Típica:
Baixa (dados efêmeros / atuais)

Exemplos de Uso Open RAN:
MySQL do 5G Core, Redis RNIB (Radio NInformation Base)

Exemplo de Consulta:
SELECT uid, imsi, state FROM subscribers WHERE ue_id=...

Caminho Crítico RAN: SIM
(Afeta o estabelecimento de chamada)



TSDB

(Banco de Dados de Séries Temporais)

Finalidade: Métricas de desempenho recentes, monitoramento.

Padrão de Escrita: Alta velocidade de ingestão de tempo-série.

Retenção Típica:
Média (dias a semanas)

Exemplos de Uso Open RAN:
InfluxDB / Prometheus – métricas recentes.

Exemplo de Consulta:
SELECT mean(thpt) FROM metrics WHERE time > now()-1h GROUP BY cell_id

Caminho Crítico RAN: NÃO
(Somente monitoramento e observabilidade)



OLAP / DATA WAREHOUSE

(Processamento Analítico Online)

Finalidade: KPIs históricos, análise de SLA, capacidade.

Padrão de Escrita:
Carga em batch (ETL/ELT)

Retenção Típica:
Alta (meses a anos)

Exemplos de Uso Open RAN:
KPIs históricos, dashboards, planejamento.

Exemplo de Consulta:
SELECT month, sum(vol_gb) FROM traffic WHERE cell_tech='5G' GROUP BY month

Caminho Crítico RAN: NÃO
(Auditoria e planejamento estratégico)

OLTP=Online Transaction Processing, TSDB=Time-Series Database, OLAP=Online-Time Analytical Processing, SLA=Service Level Agreement, ETL=Extract Transform Load, KPI=Key Performance Indicator.

Por que não consultar diretamente as NFs?

- Carga sobre funções críticas · retenção limitada
- Esquemas orientados à operação · histórico incompleto
- Dados distribuídos · timestamps inconsistentes
- Versões de software · correlação difícil
- Risco de exposição (assinantes / UE)

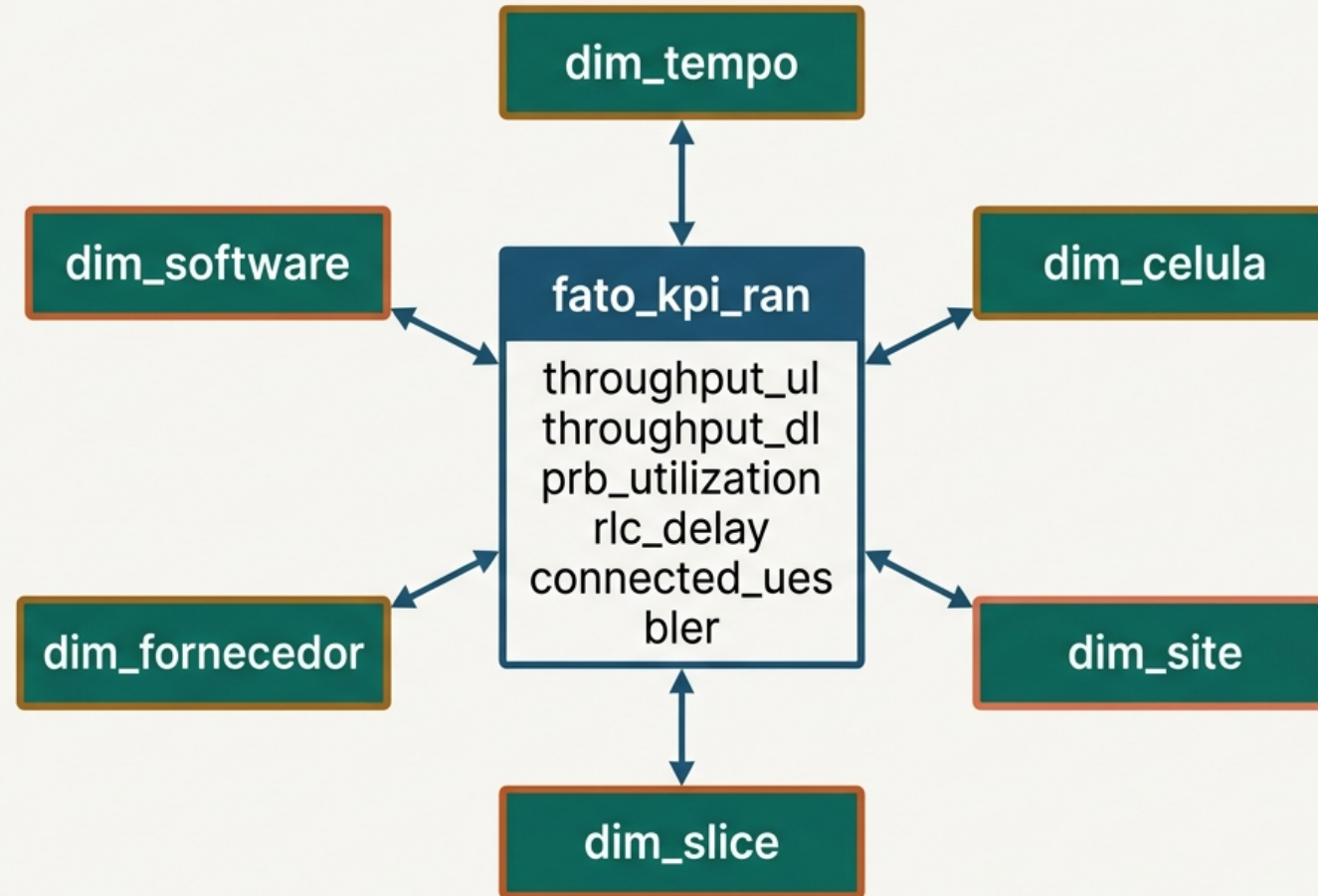
Plano operacional $\neq$ plano analítico.

Esquema estrela — KPIs de RAN

Perguntas que o modelo responde

- Célula com maior degradação no pico?
- Throughput após update de software?
- Quais slices violaram SLA com mais frequência?
- Diferença por região / fornecedor?

Esquema estrela — KPIs de RAN



Granularidade exemplo: 1 linha = célula × 15 min × slice

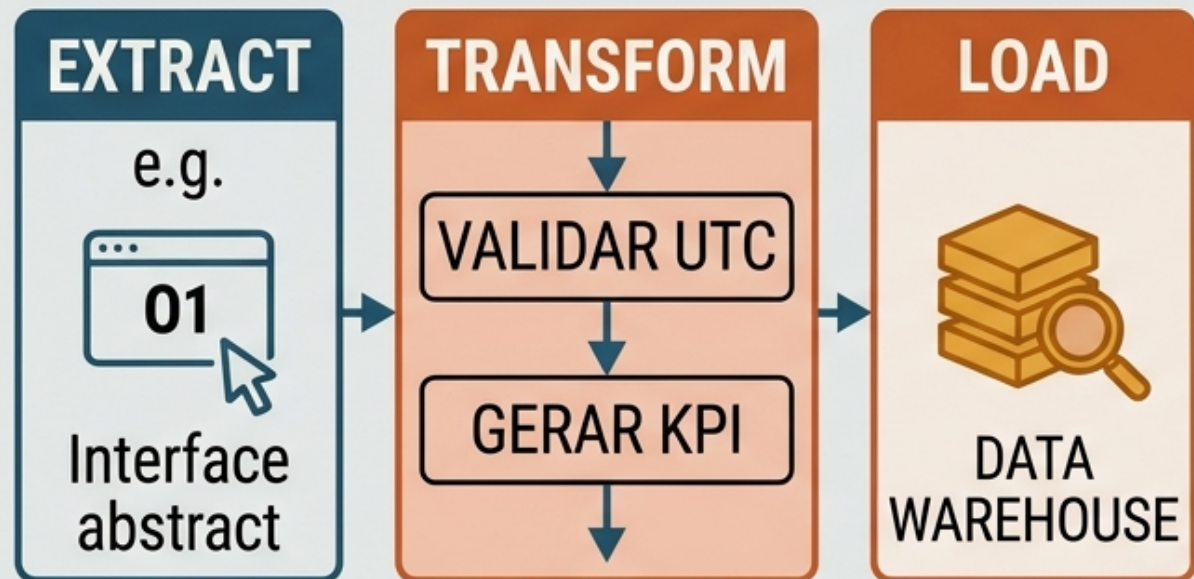
Esquema floco — KPIs de RAN

Perguntas que o esquema floco responde

- Como KPIs variam entre diferentes níveis hierárquicos? (célula → site → região)
- Qual a contribuição de cada site dentro de uma região para indicadores agregados?
- Existem padrões ou anomalias em determinados grupos de sites ou regiões?
- Como a performance se distribui por tecnologia, banda, fornecedor ao detalhar por agrupamentos?

ETL x ELT: COMPARAÇÃO EM OPEN RAN

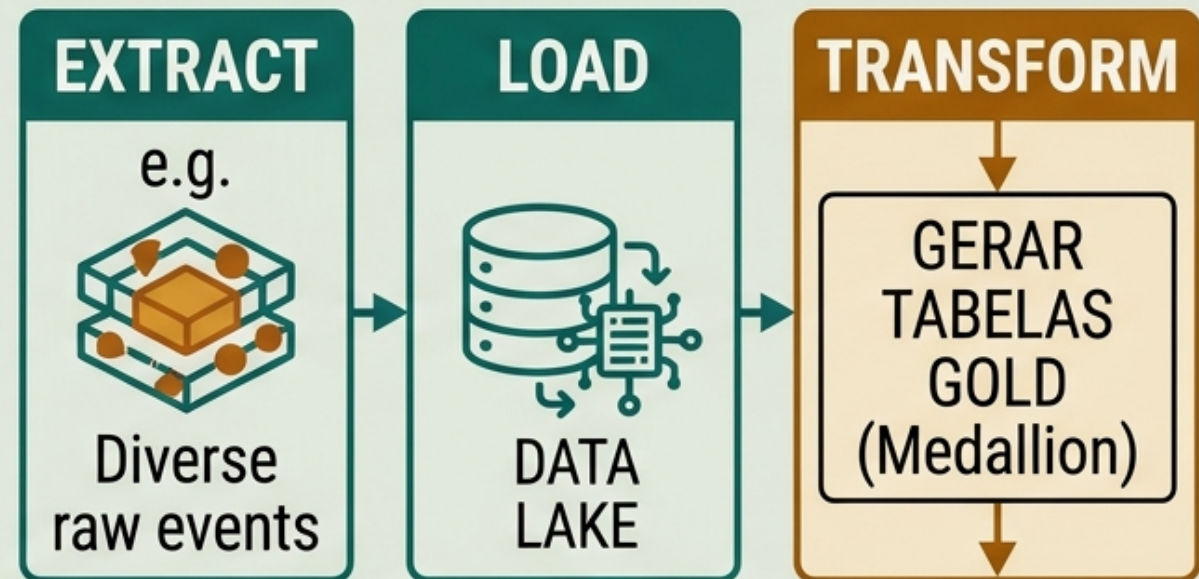
ETL (Extract -> Transform -> Load)



ALTA GOVERNANÇA NA ENTRADA

DADOS PRONTOS NO DW

ELT (Extract -> Load -> Transform)



MÁXIMA FLEXIBILIDADE

DADOS REPROCESSÁVEIS NO LAKE

ETL=Extract Transform Load, ELT=Extract Load Transform, DW=Data Warehouse
01=Interface de Gestão O-RAN (PM/FM), KPI=Key Performance Indicator, UTC=Coordinated Universal Time

ETL e ELT na RAN

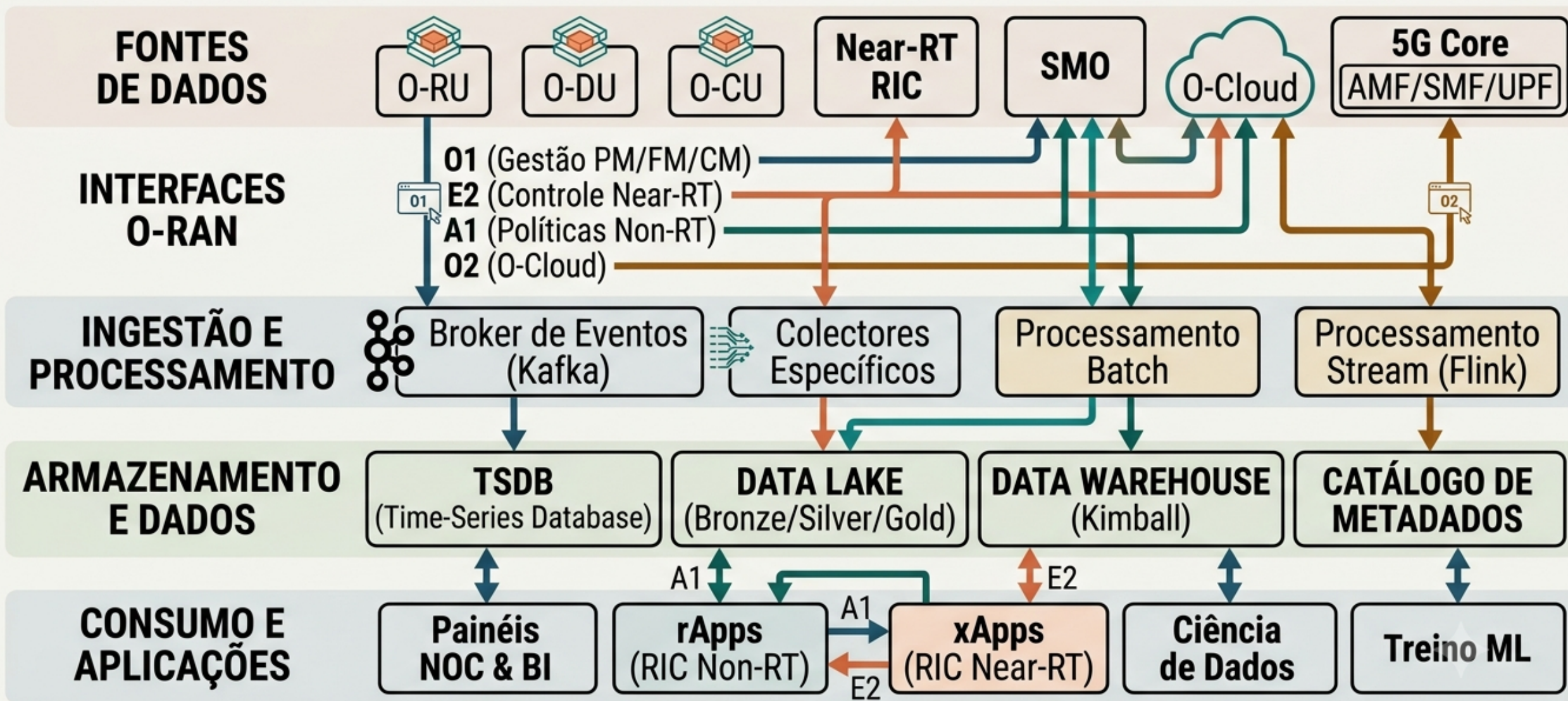
ETL — Extract → Transform → Load

coletar O1 → validar UTC → normalizar IDs → calcular KPI → carregar DW

ELT — Extract → Load → Transform

ingerir bruto no Lake → preservar origem → transformar depois → tabelas gold

ARQUITETURA ANALÍTICA INTEGRADA PARA OPEN RAN



Schema-on-write × schema-on-read

	Write (típico DW)	Read (típico Lake)
Quando define	na carga	no consumo
Validação	antecipada	sob demanda
Uso	KPIs / relatórios	exploração / bruto / pesquisa
Exemplo	tabela Thp/célula/15min	envelope E2 ou log original

Schema-on-read ≠ ausência de esquema — exige contrato, metadados e linhagem.

 Bloco 2 — bancos de dados

SQLite · MySQL · Mongo · Firebase · InfluxDB (OSC)

Famílias de bancos — mapa mental

Sistema	Modelo	Onde no lab / OSC	Papel típico
SQLite	SQL embutido	kpm.sqlite (projeto)	Offline, EDA
MySQL	SQL servidor	Core OAI mysql	OLTP assinaturas
InfluxDB	TSDB	Non-RT RAN PM (OSC)	Séries PM/KPM → rApps
Redis	KV em memória	ric_dbaas Near-RT	RNIB / SDL
MongoDB	Documentos	conceitual	JSON flexível
Firebase	BaaS NoSQL	conceitual	Apps tempo real

Subconjuntos da linguagem SQL

GRUPO	SIGNIFICADO	INTERAGE COM	COMANDOS
DDL	Data Definition Language	Objetos do banco (schema)	CREATE , ALTER , DROP
DML	Data Manipulation Language	Dados nas tabelas	INSERT , UPDATE , DELETE
DQL	Data Query Language	Consulta	SELECT
DTL	Data Transaction Language	Controle de transação	BEGIN TRANSACTION , COMMIT , ROLLBACK
DCL	Data Control Language	Segurança / permissões	GRANT , REVOKE , DENY

Nota: em alguns livros o `SELECT` fica na DML; em outros, no grupo próprio DQL. No lab: DDL/DML no store · DQL nas demos SQLite/MySQL.

SQLite — o banco do projeto

- **Zero servidor:** um arquivo; ideal para artefato docente e notebooks.
- ACID em um processo; escrita concorrente limitada (não é cluster).
- No lab: `kpm.sqlite` com `runs` + `kpm_samples` (`kpm_store.py`).
- **Quando usar:** trilha offline, EDA local, demos reprodutíveis.
- **Quando não:** muitos escritores simultâneos, multi-tenant de produção.

SQLite — schema do lab (real)

```
runs(run_id, created_at, use_case, notes)
kpm_samples(id, run_id, phase, sample_index,
            ingested_at, source_path, payload_json)
```

- `phase` : `baseline` · `stress` · `recovery`
- `payload_json` : métricas (UEThp, PRB, delay...) em JSON
- Espelho bronze: `kpm.jsonl` (uma linha = um registro)

MySQL — SQL com servidor

- Cliente/servidor; forte em **OLTP** e consistência relacional.
- No lab OAI: container `mysql` guarda assinaturas 5G (`oai_db`).
- **Não confundir** com warehouse: aqui serve o plano de controle (UE/IMSI), não KPI histórico.
- **Quando usar:** apps com tabelas/relacionamentos, multi-usuário, backups/replicação.
- **Quando não:** dumps brutos de PM (prefira lake/colunar).

MongoDB — documentos

- Documentos BSON/JSON; schema flexível (evolui com o produto).
- Bom para eventos semiestruturados, catálogos, payloads heterogêneos.
- Escala horizontal (sharding); consultas por campos aninhados.
- **Analogia telco:** armazenar alarmes FM ou envelopes KPM como documentos.
- **Cuidado:** sem disciplina vira “JSON bagunçado” — ainda precisa de contrato de dados.

Firebase — BaaS (Firestore / RTDB)

- Plataforma gerenciada: auth, sync tempo real, regras de segurança, SDKs mobile/web.
- Firestore: documentos/coleções; RTDB: árvore JSON sincronizada.
- **Bom para:** apps colaborativos, dashboards leves, protótipos rápidos.
- **Ruim como “warehouse telco”:** não substitui lake de PM, retenção longa nem BI corporativo.
- No módulo: conceito + CLI; **não** é o store do projeto avaliativo.

InfluxDB — TSDB do O-RAN SC (Non-RT)

- **Time-series DB:** otimizado para métricas com timestamp (PM/KPM).
- No OSC Non-RT RAN PM: *Influx Logger* consome Kafka e grava em InfluxDB.
- rApps (UE-TP, Cell-TP, analytics) **leem** séries do Influx — base NGO §6.1.
- Modelo: *measurement* (recurso) · *fields* (contadores) · *tags* · tempo = fim do período.
- **Quando usar:** telemetria contínua, janelas, painéis Grafana, features para ML.

Caminho de dados OSC — VES → Influx

1. RAN publica disponibilidade de arquivo PM (evento **VES**).
2. **Data File Collector** busca XML (3GPP TS 32.432/435) → conversão JSON.
3. **PM Producer** filtra/distribui via Kafka; rApps assinam tipos de medição.
4. **Influx Logger** persiste contadores selecionados no **InfluxDB**.
5. rApp consulta Influx (Flux/InfluxQL) para previsão / política A1.

Refs.: docs [O-RAN SC Non-RT RAN PM](#) *nonrttric-plt-ranpm* / Influx Logger · Minio (arquivos) + Kafka (eventos).

Redis `ric_dbaas` ≠ InfluxDB

	Redis (dbaas)	InfluxDB
Onde	Near-RT plataforma	Non-RT RAN PM / analytics
Dado	RNIB: E2 nodes, subscrições	Séries PM/KPM no tempo
Pergunta	“Quais gNBs estão up?”	“UEThp nas últimas 2 h?”
Lab	✓ container <code>ric_dbaas</code>	referência OSC + demo local
Projeto	diagnóstico E2	analogia: SQLite/JSONL ou TSDB

Comparativo — qual escolher?

Critério	SQLite	MySQL	InfluxDB	Redis	Mongo/Firebase
Modelo	SQL arquivo	SQL OLTP	TSDB	KV	docs / BaaS
Consulta	SQL	SQL	Flux/InfluxQL	KEYS/GET	SDK/MQL
Lab M09	✓ amostra	✓ Core	OSC / demo	✓ dbaas	conceitual
Projeto	✓ padrão	opcional	se justificar	diagnóstico	raro

Exercício guiado

1. Artefatos com KPM para EDA → **SQLite (+ JSONL)**
2. AMF valida IMSI no attach → **MySQL (Core)**
3. rApp Non-RT lê PM contínuo (OSC) → **InfluxDB** (via Influx Logger)
4. xApp pergunta se o gNB registrou no Near-RT → **Redis ric_dbaas**
5. App NOC mobile sync → **Firebase** (UI) · eventos FM flexíveis → **Mongo / lake**

 **Bloco 3 — 20:10–21:00**

Terminal: consultar e inspecionar dados

Ambiente do terminal (caminhos)

```
cd modulo09-analise_dados_redes_telecom
ls code/datasets/kpm-ue-tp-sample/          # amostra offline

cd code/oai-cn-gnb-nonrt-nearrt
docker ps --format '{{.Names}}' | grep -E \
'mysql|ric_dbaas|nonrt-policy|influx'
```

Pré-requisitos: `sqlite3` + `docker` . Fase 2: Near-RT OSC + Non-RT PMS. Influx completo = stack RAN PM (fora do compose slim).

Demo 1a — SQLite na amostra do projeto

```
cd code/datasets/kpm-ue-tp-sample

sqlite3 kpm.sqlite ".tables"
sqlite3 kpm.sqlite ".schema kpm_samples"

sqlite3 -header -column kpm.sqlite \
    "SELECT phase, COUNT(*) AS n
    FROM kpm_samples GROUP BY phase;"

sqlite3 -header -column kpm.sqlite \
    "SELECT phase, sample_index,
    json_extract(payload_json, '$.DRB.UETHpU1') AS ue_thp_u1
    FROM kpm_samples
    WHERE phase='stress' LIMIT 5;"
```

Demo 1b — JSONL (bronze) no terminal

```
# Contar linhas / ver uma amostra
wc -l kpm.jsonl
head -n 1 kpm.jsonl | python3 -m json.tool | head -40

# Extrair fase com jq (se instalado)
jq -r '.phase' kpm.jsonl | sort | uniq -c

# Via Python (sempre no lab)
python3 - <<'PY'
import json
from collections import Counter
from pathlib import Path
c=Counter(json.loads(l)["phase"] for l in Path("kpm.jsonl").open())
print(c)
PY
```

Demo 1c — pipeline do lab (regenerar)

```
cd code/oai-cn-gnb-nonrt-nearrt

# Offline: gera logs/experiments/ue-tp-<ts>/
./scripts/run_ue_tp_experiment.sh

# Inspecionar DB gerado
DB=$(ls -1dt logs/experiments/ue-tp-*/kpm.sqlite | head -1)
sqlite3 -header -column "$DB" \
    "SELECT phase, COUNT(*) FROM kpm_samples GROUP BY phase;"
```

Para avaliação, preferir a amostra versionada em `code/datasets/kpm-ue-tp-sample/`.

Demo 2 — MySQL do Core OAI (Docker)

```
# Subir Core (se necessário)
./scripts/up_core.sh    # ou já estar com oai-amf/mysql up

docker exec -it mysql mysql -u test -ptest oai_db -e "SHOW TABLES;"

docker exec -it mysql mysql -u test -ptest oai_db -e \
    "SELECT ueid, servingPlmnid FROM AccessAndMobilitySubscriptionData LIMIT 5;"

docker exec -it mysql mysql -u test -ptest oai_db -e \
    "SELECT ueid FROM AuthenticationSubscription LIMIT 5;"
```

Objetivo didático: ver **OLTP do core** ≠ store analítico de KPM.

Demo 3 — Redis RNIB (**ric_dbaas**, Near-RT OSC)

```
# Requer Fase 2 Near-RT up (./scripts/up_oran_ric.sh ...)  
docker exec ric_dbaas redis-cli INFO server | grep redis_version  
  
docker exec ric_dbaas redis-cli KEYS '{e2Manager},RAN:gnb_*'  
  
docker exec ric_dbaas redis-cli \  
  KEYS '{e2Manager},RAN:gnb_*' | head -1 | xargs -I{} \  
  docker exec ric_dbaas redis-cli TYPE '{}'
```

Prova: inventário E2 no Near-RT. **Não** substitui Influx/SQLite para histórico de vazão.

Demo 4a — MongoDB (opcional / conceitual)

```
# Exemplo local com Docker (não faz parte do compose OAI padrão)
docker run -d --name mongo-demo -p 27017:27017 mongo:7

docker exec -it mongo-demo mongosh --eval '
  db.kpm.insertOne({
    phase: "stress",
    metrics: {"DRB.UEThpUL": 80000, "RRU.PrbTotUL": 97}
  });
  db.kpm.find().pretty();
'

docker rm -f mongo-demo
```

Use para sentir o modelo documento; o projeto continua em SQLite/JSONL.

Demo 4b — Acessando dados do core Open5GS via MongoDB

```
# Inspecionar dados reais gravados pelo núcleo 4G/5G do Open5GS (MongoDB)

# Listar os bancos de dados usados pelo Open5GS
docker exec -it open5gs-mongodb-containerized mongosh --eval "show dbs"

# Entrar no shell do MongoDB para explorar as coleções criadas pelo core
docker exec -it open5gs-mongodb-containerized mongosh

# Exemplos de comandos dentro do mongosh:
# 1. Usar banco do Open5GS
use open5gs

# 2. Listar coleções (tabelas) típicas do Open5GS
show collections
```

Demo 4b — Acessando dados do core Open5GS via MongoDB

```
# 3. Consultar informações dos assinantes (subscriber)
db.subscribers.find().pretty()

# 4. Consultar sessões de contexto de usuário (p.ex., 'session')
db.session.find().pretty()

# 5. Buscar dados do AMF / SMF (geralmente em coleções como 'amf', 'smf')
db.amf.find().limit(2).pretty()
db.smf.find().limit(2).pretty()
```

O **Open5GS** utiliza MongoDB para armazenar informações do núcleo do core, como assinantes, contexto de sessão, e dados de gerenciamento. Esses exemplos ajudam a visualizar o modelo documental e a acessar dados reais.

Mapa mental — comando → objetivo

Comando	O que prova
<code>sqlite3 ... GROUP BY phase</code>	Silver/gold local (projeto)
<code>head kpm.jsonl json.tool</code>	Bronze / schema-on-read
<code>docker exec mysql ...</code>	OLTP Core ≠ analytics
<code>redis-cli KEYS ...gnb_*</code>	RNIB Near-RT (Redis)
<code>curl .../api/v2/query</code> (Flux)	TSDB estilo Influx / OSC
<code>mongosh</code> / <code>firebase</code>	Docs / BaaS (outros papéis)

Discussão guiada

1. Por que Redis `ric_dbaas` não serve para treinar UE-TP com 2 h de KPM?
2. No OSC, onde entra Influx vs Minio (arquivos PM XML)?
3. Projeto M09: quando SQLite basta e quando faria sentido um TSDB?

Checkpoint — caso, hipótese e arquitetura (20–30 min)

- **Entregar:** tema G1–G7 + pergunta + uso de PRB/delay/vazão + 2 indicadores.
- **Arquitetura:** bronze · silver/gold · papel de SQLite vs Influx (OSC) vs Redis RNIB.
- **Insumo:** `docs/temas-grupos.md` + `code/datasets/kpm-ue-tp-sample/`.
- **Prova:** SQL na amostra *e/ou* `KEYS gnb_*` no `ric_dbaas` (se Fase 2 up).

Síntese e verificação

- Lake absorve bruto; warehouse/gold sustenta KPI; **InfluxDB** é o TSDB do caminho OSC Non-RT (PM → rApps).
- SQLite = projeto · MySQL = Core · Redis = RNIB Near-RT · Mongo/Firebase = outros papéis.
- **Verifique:** hipótese + mapa SQLite↔Influx↔Redis + um comando que roda (SQL ou `KEYS gnb_*`).

Plataforma Web com exercícios propostos

Plataforma unificada de atividades (Módulos 05 · 07 · 09):

 Exercícios desta aula (Dados Telco) →



Dados Telco

·



O-RAN

·



RIC

·



Hub

Preparação — Aula 03 (Checkpoint 1)

08/08/2026 — EDA, ETL e visualização (5 h)

- Trazer a proposta de arquitetura (incluindo escolha de DB)
- Python/pandas + SQLite; amostra `kpm-ue-tp-sample`
- Leitura: Wong Cap. 5 (data pipeline) · Tripathi Cap. 3



Referências

TRIPATHI, N. D.; SHAH, V. K. *Fundamentals of O-RAN*. Wiley-IEEE Press, 2025.

WONG, I. C. et al. (Eds.) *Open RAN: The Definitive Guide*. Wiley-IEEE Press, 2024. (Caps. 3–4)

YANOVER, V. (Ed.) *Open RAN: Technology and Ecosystem*. Wiley, 2025.

Databricks — *Data Lakes vs Data Warehouses* (visão lakehouse).



Referências complementares

O-RAN SC Non-RT RAN PM / Influx Logger — [docs.o-ran-sc.org ... nonrt-ric-plt-ranpm](https://docs.o-ran-sc.org/en/Contrib/infrastructure/Non-RT-RAN-PM/)

SQLite · MySQL · InfluxDB · MongoDB · Firebase docs oficiais.

Lab: `kpm_store.py` · MySQL `oai_db` · Redis `ric_dbaas` · Non-RT compose slim (sem Influx)

NGO §6.1 / M07: VES → InfluxDB → rApps (UE-TP / Cell-TP) · FlexRIC iApp `influx`



Jonas Augusto Kunzler



Email: jak@cesar.school



LinkedIn: [jakunzler](#)



GitHub: [jakunzler](#)



Instagram: [@jakunzler](#)